---

title: 'TSCP: Cryptographic Custody Verification for Multi-Agent Computation Pipelines' summary: Draft v2 — August 2026

TSCP: Cryptographic Custody Verification for Multi-Agent Computation Pipelines

Draft v2.1 — August 2026

Abstract

We present the Triune Structured Codex Protocol (TSCP), a protocol for cryptographic custody verification of artifacts produced by multi-agent computation pipelines. TSCP introduces a five-stage custody pipeline — Resolve, Observe, Bind, Independent Verify, Promote — that is designed to transform declared hashes from assertions into independently verified claims. The protocol is built on a categorical formal backbone in Lean 4, proving that verification properties (admissibility, preservation, reflection) compose correctly across systems. We demonstrate the protocol's implementation on the Plonky3 proof system, including an AVX-512 number-theoretic transform (NTT) optimization achieving 9.15x speedup with 61 verification points, backed by 83 formal theorems and 104 test vectors. We further present results from an adversarial discovery audit (ARCHER methodology) identifying 36 boundary-level findings, including authorization, input-validation, integrity, serialization, and incomplete-verifier defects. Findings were either fixed, explicitly contained, or documented as open engineering boundaries. A separate binding audit found that the proof system does not yet cryptographically bind an external artifact to its claimed computation — the central open requirement. The protocol's core principle — "a declared hash is an assertion, not a verification" — distinguishes it from existing approaches that treat cryptographic hashes as sufficient evidence of provenance.

Important. The current implementation should be regarded as a research prototype rather than a production ZK custody verifier. Several verifier and on-chain components remain incomplete and are therefore excluded from the protocol's unconditional security claims. In particular, the protocol defines artifact binding (Bound(A, C, W) => A = Canonicalize(Output(C))) as a central requirement, but the current implementation does not yet establish this binding — the proof system proves trace properties, not artifact provenance. This paper documents precisely what is proven, what is implemented, and what remains open.

1. Introduction

1.1 The Problem

As AI systems increasingly produce artifacts that downstream systems depend on — code, decisions, data, model weights, proofs — there is no standard mechanism to verify that a claimed artifact actually passed through its claimed computation pipeline. Current approaches fall into three categories:

> **Trust declared hashes.** A system produces an artifact, computes its hash, and publishes it. Downstream systems compare hashes to confirm identity. This is the dominant approach in software supply chains (e.g., SBOM, SLSA provenance). It is unsafe: a hash confirms *what* the artifact is, not *how* it was produced.

Full recomputation. Downstream systems re-run the computation to verify the output. This is safe but expensive — it defeats the purpose of delegation and does not scale to complex pipelines.

No verification. Most multi-agent AI systems today simply trust the claimed output of upstream agents. This is the common case, and it is the case TSCP is designed to address.

1.2 The Gap

The gap between these approaches is what we call *custody verification*: a structured process for verifying that an artifact's claimed provenance — the sequence of computations that produced it — is real, without requiring full recomputation. Zero-knowledge proofs provide a tool for this: a prover can demonstrate that a computation was executed correctly without revealing the computation's inputs or requiring the verifier to re-execute it. But existing ZK proof systems prove *computation correctness*, not *artifact provenance*. They answer "was this computation valid?" but not "did this artifact actually come from this computation, and is the chain of custody intact?"

1.3 Our Contribution

TSCP provides a formal custody framework that separates artifact identity, artifact resolution, and artifact-to-computation binding into distinct predicates, models custody promotion as a sequence of predicate-dependent state transitions, and provides a mechanically verified composition theorem for certified verification bridges. It is not a new hash function, a new FRI protocol, a new ZK proof system, a replacement for SLSA, a complete production ZK verifier, or a proof that arbitrary AI outputs are truthful.

The contribution has three axes:

Predicate separation: Artifact identity (H(A) = h), resolution (Resolve(h) = A), and provenance binding (A = Canonicalize(Output(C))) are distinct properties that existing systems frequently conflate. TSCP makes them explicit.

Promotion semantics: Custody promotion is modeled as a sequence of predicate-dependent state transitions (Section 3.1.1), where promotion requires all preceding predicates to hold. Failed predicates cannot result in promotion.

Certified composition: A mechanically verified composition theorem (Lean 4) proves that certified bridges preserve admissibility, preservation, and reflection properties across verification systems (Section 3.2).

Additionally, TSCP provides:

An experimental ZK implementation (Section 3.3) on the Plonky3 proof system, including a sumcheck protocol with self-checking prover, FRI commitment scheme, and OWSL execution-environment health gating. Several verifier components remain incomplete (Section 3.6).

An NTT optimization (Section 3.4) using AVX-512 intrinsics, achieving 9.15x speedup over the scalar baseline, with formal verification of the mathematical NTT specification and conditional refinement of the AVX-512 implementation to that specification.

None of these contribution axes requires pretending that artifact binding has already been solved. However, the novelty of the custody-state composition depends on establishing that it provides a

technically necessary property absent from prior systems. Existing supply-chain attestation systems (SLSA, in-toto) bind artifacts to provenance metadata through trusted builders; proof-carrying data systems provide recursive proof composition; ZK proof systems prove computation correctness. TSCP's proposed contribution is the specific composition — separating identity, observation, binding, verification, and promotion into a fail-closed state machine — rather than any individual component. Whether this composition is technically novel is a question for future comparison against these systems at the same abstraction level (Section 6.2-6.3).

1.4 The Core Principle

TSCP is organized around a single principle: **a declared hash is an assertion, not a verification.** Most systems treat a published hash as proof that an artifact is what it claims to be. TSCP treats it as a *claim* that must be independently verified through the custody pipeline before it can be promoted to a verified fact.

This principle is not merely philosophical. It has concrete security consequences: the ARCHER audit (Section 4) found that the protocol's implementations initially treated several boundary conditions — empty predecessors, default authorization scopes, placeholder verification flags — as verified when they were merely asserted. Each of these was a case where a declared hash (or its equivalent) was being treated as a verification.

1.5 Artifact Binding: The Central Predicate and Its Current Status

The protocol's central contribution is making artifact binding mathematically explicit. We define a formal predicate:

Bound(A, C, W)

where:

A = canonical artifact (resolved to canonical bytes)

C = computation claim (committed inputs, program, execution environment)

W = witness/proof

and require:

Bound(A, C, W) => A = Canonicalize(Output(C))

subject to the cryptographic and implementation assumptions stated in Section 3.6.

Current status: this binding is a protocol requirement, not yet satisfied by the implementation. A binding audit (Section 4.5) found that the current proof system proves properties about trace columns (sumcheck over the folded oracle) but does not cryptographically bind an external artifact A to the computation output Output(C). The proof request takes raw trace columns as input with no artifact hash; the proof envelope seals a claim value and proof payload with no artifact reference; the on-chain anchor commits a batch hash that is not linked to the proof. We have:

```
H(A) = h  (identity)
Verify(pi, C) = true  (proof of trace properties)
```

but no established relation:

```
Verify(pi, C, h_A) => h_A = H(Output(C))
```

This is the central open problem. Making it explicit — rather than assuming it is implicitly satisfied — is itself a contribution of this paper.

This is distinct from three weaker predicates that TSCP explicitly separates:

Identity: H(A) = h — the artifact hashes to the declared value.

Resolution: Resolve(h) = A — the artifact can be retrieved and canonicalized.

Provenance binding: ProofVerify(pi, A, C) = true — the artifact is cryptographically bound to the claimed computation.

The custody pipeline is designed to ensure all three hold before promotion. In the current implementation, only identity and resolution are established. Provenance binding is the protocol's central open requirement. The distinction between identity, resolution, and provenance binding is one of the paper's contributions: existing systems frequently conflate the first with the third.

2. Background

2.1 Zero-Knowledge Proof Systems

A zero-knowledge proof system allows a prover to convince a verifier that a statement is true without revealing any information beyond the validity of the statement. Modern ZK proof systems based on the FRI protocol (Fast RS Codes Interactive Oracle Proof of Proximity) work as follows:

Commit phase. The prover commits to a polynomial via a Merkle tree of evaluations over a dedicated domain. The commitment is a Merkle root, which the verifier can check against later.

Challenge phase. Challenges are derived from the commitment via a Fiat-Shamir transform, making the protocol non-interactive. The prover cannot adapt its committed values after seeing the challenges.

Query phase. The verifier samples query indices and checks that the prover's openings at those indices are consistent with the claimed polynomial. Each round's fold consistency is verified: the folded value at the next round must equal the even/odd decomposition of the current round's values, weighted by the challenge.

Final check. After log2(n) folds, the polynomial reduces to a single constant. The verifier checks that this constant matches the claimed final value.

The soundness of FRI depends on the prover being committed to its evaluations before learning which indices will be queried. If the prover could choose evaluations after seeing the queries, it could cheat at unchecked points.

2.2 Plonky3

Plonky3 is a modular ZK proof system developed by Polygon. It provides the building blocks used by TSCP:

BabyBear field (p = 2^27 - 2^24 + 1), a 31-bit prime field used by Plonky3 — the base field for all arithmetic

Poseidon2 permutation — the hash function used for Merkle commitments and Fiat-Shamir challenges

MerkleTreeMmcs — the Merkle matrix commitment scheme (MMCS) for vector commitments

DuplexChallenger — the Fiat-Shamir transcript for deriving challenges

TSCP uses Plonky3's MMCS for all Merkle commitments, rather than a hand-rolled hash tree. This is a deliberate design choice: the commitment scheme is the trust root of the entire protocol, and using a well-tested implementation reduces the attack surface.

2.3 Multilinear Extensions and Sumcheck

A multilinear extension (MLE) of a function f: {0,1}^n -> F is the unique multilinear polynomial that agrees with f at all Boolean points. MLEs are central to modern ZK proof systems: the sumcheck protocol allows a prover to convince a verifier of the sum of an MLE over the Boolean hypercube, with the verifier only needing to evaluate the MLE at a single random point.

TSCP uses sumcheck to prove that the constraint evaluations over the trace are consistent with the claimed constraints. The prover computes the MLE of the constraint evaluations and runs the sumcheck protocol; the verifier checks each round's claimed sum against a fresh challenge.

2.4 Number-Theoretic Transform

The NTT is a fundamental building block for polynomial evaluation in FRI-based proof systems. It evaluates a polynomial at all points of a multiplicative subgroup in O(n log n) operations. The NTT is a primary computational kernel in polynomial evaluation for STARK/FRI architectures, making it an important kernel optimization target.

AVX-512 instructions provide 512-bit SIMD operations that can process 16 BabyBear elements (32-bit each) simultaneously. The butterfly operation at the heart of the NTT — computing (a + b, (a - b)*w) for elements a, b and twiddle factor w — maps naturally to SIMD lanes.

2.5 Lean 4

Lean 4 is an interactive theorem prover and programming language. TSCP uses Lean 4 for two purposes:

1. **Formal verification of the protocol's structure** — the categorical backbone (Kernel, Universe, Bridge) and the composition theorem
2. **Formal verification of the mathematical NTT specification** — proving the mathematical correctness of Montgomery multiplication, butterfly operations, and stage decomposition, with conditional refinement of the AVX-512 implementation

Lean 4's dependent type theory provides strong guarantees: a theorem proved in Lean is checked by a small, trusted kernel, and the proof is machine-verifiable. Where machine-level refinement is not yet available, we explicitly document open axioms rather than claiming full implementation verification.

3. The TSCP Protocol

3.1 The Custody Pipeline

The custody pipeline is the core operational component of TSCP. It defines five stages through which an artifact must pass before its provenance can be considered verified:

Stage 1: Resolve. Given an artifact's declared hash, resolve it to its canonical representation. This includes resolving symbolic references, canonicalizing serialization, and checking that the artifact exists at the claimed location. The resolution stage ensures that all downstream stages operate on the same byte-level representation.

Stage 2: Observe. Record the artifact's observable properties — its hash, size, type, and the computation environment that claims to have produced it. Observation does not verify; it records what can be independently checked. The OWSL (Oracle Witness Security Ledger) component monitors execution-environment health during this stage, gating further processing on entropy availability (Section 3.3.3).

Stage 3: Bind. Is designed to cryptographically bind the artifact to its claimed computation via the predicate Bound(A, C, W) (Section 1.5). **Note: this stage is not yet implemented — the current proof system proves trace properties, not artifact binding (Section 4.5).** When implemented, this will be a sumcheck proof combined with a FRI commitment that includes the artifact hash as a public input. The binding stage is designed to produce a proof artifact that can be independently verified.

Stage 4: Independent Verify. Independently verify the proof's cryptographic correctness. When binding is implemented, this stage verifies the binding proof. This stage does not trust the prover; it reconstructs an independent transcript from the same committed public values and deterministically re-derives the Fiat-Shamir challenges. In the oracle-layer FRI implementation, all Merkle openings are checked and fold consistency is verified at each FRI round. (The delta_fri_bridge is currently a scaffold.)

Stage 5: Promote. If and only if all four preceding stages succeed, promote the artifact from "asserted" to "verified" status. The promotion is recorded on-chain (currently Sepolia testnet) via the TSCPAnchor contract, which prevents duplicate anchoring and records the committer's identity. The on-chain anchor is the *result* of promotion, not a substitute for it — anchoring a hash does not establish computational provenance.

The pipeline is sequential and fail-closed: any stage's failure prevents promotion. No stage promotes an earlier stage's claim merely because it was declared successful; each stage evaluates the predicates required for its own acceptance.

3.1.1 Custody State Machine

The pipeline admits a formal state machine:

```
A_0 = Asserted
Resolve(A_0) -> A_1
Observe(A_1) -> A_2
Bind(A_2) -> A_3
Verify(A_3) -> A_4
Promote(A_4) -> Verified
```

with the promotion invariant:

```
Promote(x) <=> Resolve(x) AND Observe(x) AND Bind(x) AND Verify(x)
```

and the failure property:

```
Failure_i => NOT Promote
```

for any stage i in {1, 2, 3, 4}.

Status: The fail-closed promotion principle is a design invariant. The implementation enforces stage sequencing and fail-closed behavior. However, because the Bind and Verify predicates are not yet complete (Sections 3.6, 4.5), the implemented promotion path does not yet enforce the full mathematical invariant — it enforces the stages that exist, with incomplete predicates in the binding and verification stages.

3.2 Categorical Formal Backbone

The formal backbone models the protocol's verification structure using category-theoretic primitives. We emphasize that the categorical structure is embodied by composable universes and certified bridges rather than constructing a formal category in the mathematical sense. We describe it as a *categorical verification backbone*.

Kernel. A kernel K over type alpha is a structure with an admissibility predicate `admits_proof: alpha -> Prop`, a proof that the predicate is decidable, and a proof that at least one proof is admissible. Kernels are the trust roots: they define what counts as a valid proof in a given system.

Universe. A universe U is a structure with three kernels — one for proofs, one for formulas, one for executions — representing the three layers of a verification system. A universe captures what can be proven, what can be formulated, and what can be executed in a given system.

Bridge. A bridge f: U -> V is a structure with maps between the proof, formula, and execution types of two universes. A bridge represents a translation or encoding between systems.

Bridge Certificate. A certificate for a bridge proves three properties: *preservation* (if p is admissible in U, then f(p) is admissible in V), *reflection* (if q is admissible in V, then some p exists with f(p) = q and p is admissible in U), and *admissibility* (the kernel admissibility structure is preserved).

Composition Theorem (mechanically proven in Lean 4). Given certified bridges f: U -> V and g: V -> W, the composite g . f: U -> W admits a certificate whose preservation, reflection, and admissibility components are constructed from the corresponding certificates of f and g.

The proof of the composition theorem is constructive and checked by Lean 4's trusted kernel. It depends on no axioms beyond the bridge certificates themselves.

3.3 ZK Implementation

The following components are experimental implementations. Several are incomplete; see Section 3.6 for a precise status table.

3.3.1 Sumcheck with Self-Checking Prover

TSCP's prover implements a sumcheck protocol that self-verifies before emitting a proof. The prover runs the sumcheck computation, then independently reconstructs a verification transcript from the same committed public values and deterministically re-derives the Fiat-Shamir challenges. If the self-check fails, the prover returns an error instead of emitting a potentially invalid proof. This is a defense-in-depth measure: even if the prover's implementation has a bug, the self-check catches it before the proof reaches a verifier.

The self-check is not a substitute for verifier-side verification. It is a prover-side integrity check that reduces the probability of emitting a malformed proof.

Status: The sumcheck prover is implemented. The sumcheck verifier is not implemented (Finding 20, Section 4.2).

3.3.2 FRI Commitment Scheme

The FRI implementation follows the standard commit-query-verify structure:

Commit. The prover builds a Merkle tree of polynomial evaluations, observes the root in the Fiat-Shamir transcript, samples a challenge, folds the evaluations in half, and repeats. After log2(n) rounds, a single constant value remains.

Query. After the full commitment, the prover samples query indices from the transcript and produces Merkle opening proofs for each index at each round. The query phase occurs only after all commitments are in the transcript, ensuring the prover cannot adapt to the queries.

Verify. The verifier reconstructs an independent transcript from the same committed public values, deterministically re-derives the challenges and query indices, then checks each round's fold consistency: the opened value at the next round must equal the even/odd decomposition of the current round's opened values, weighted by the round's challenge. All Merkle openings are verified against the claimed roots using Plonky3's MMCS.

Status: The oracle-layer FRI implementation (fri_query.rs) passed the ARCHER tests for Fiat-Shamir transcript reconstruction, Merkle verification, and fold consistency. This testing does not establish cryptographic soundness. The delta_fri_bridge.rs is a scaffold that unconditionally returns success and does not perform real FRI verification (Finding 14). Connecting the DEEP-ALI quotient to the real FRI prover is identified as future work.

3.3.3 OWSL Execution-Environment Health Gating

The OWSL (Oracle Witness Security Ledger) component is an execution-environment health gate intended to detect compromised or degraded randomness infrastructure relevant to cryptographic operations. If available entropy falls below a threshold (default: 256 bits), the system enters a WARNING state; below 128 bits, it enters a CRITICAL state and stops processing.

We note that Fiat-Shamir challenges are derived deterministically from the transcript. The relevant security question is the unpredictability and programmability model of the transcript construction, not simply whether the kernel reports a specific entropy level. OWSL is therefore best understood as a defense against degraded execution environments, not as a proof that low entropy breaks Fiat-Shamir soundness.

The OWSL status includes a SHA-256 content hash over all status fields, verified by the Rust bridge on read. This prevents a compromised daemon from falsifying its own health reports without detection.

3.4 NTT Optimization

The AVX-512 NTT implementation processes 16 BabyBear elements per butterfly operation using 512-bit SIMD lanes. The formal verification in Lean 4 establishes the following chain:

Mathematical layer (fully proven): Montgomery multiplication is correct (`montgomeryMul_scalar_correct`); the NTT butterfly decomposition is correct; the stage decomposition is correct.

Machine layer (3 open axioms): The AVX-512 intrinsics (`exec_avx512_mont_mul`, `exec_avx512_butterfly`) are axiomatized as refinements of the scalar algorithms. These are machine refinement obligations — the gap between the mathematical specification and the hardware implementation. They are not proven; they are formalized as axioms backed by empirical testing.

Connection (proven from axioms): By transitivity, the AVX-512 implementation equals the mathematical specification: `exec_avx512_mont_mul = scalarMul (axiom) = montgomeryMul (proven)`.

We therefore describe this as *formal verification of the mathematical NTT specification and conditional refinement of the AVX-512 implementation to that specification*, not as full formal verification of the AVX-512 implementation. The three open axioms are formalized refinement obligations, not discharged theorems.

The 104 Rust test vectors (Section 3.7) verify that the AVX-512 output matches the scalar implementation across boundary cases, random inputs, and round-trip property tests. Discharging the machine axioms via machine-level verification (e.g., Aeneas or Kraken) is identified as future work.

3.5 Formal Axiom Dependency Table

The formal verification has five explicitly documented axioms. The dependency structure is:

Layer	Result	Status	Assumptions
Montgomery arithmetic	Correctness	Proven	None
Butterfly	Correctness	Proven	None
Stage decomposition	Correctness	Proven	None
AVX-512 refinement	Machine equivalence	Conditional	3 machine axioms
NTT bridge	Preservation	Conditional	execution_valid
End-to-end NTT	Preservation	Conditional	babybear_ntt_end_to_end
Bridge composition	Composition	Proven	None beyond certificates

The three machine refinement axioms are:

1. `exec_avx512_mont_mul` — AVX-512 Montgomery multiplication equals scalar Montgomery multiplication
2. `exec_avx512_butterfly` — AVX-512 butterfly equals scalar butterfly
3. AVX-512 lane packing correctness

The two engineering axioms are:

1. `execution_valid`: The NTT bridge has a valid certificate (the NTT preserves admissibility). Evidence: AVX-512 butterfly kernel implementation + test suite.
2. `babybear_ntt_end_to_end`: The NTT forward transform preserves admissibility end-to-end. Evidence: NTT round-trip test vectors.

All axioms are documented in the source with their evidence basis. A reader of the formal layer can identify exactly which theorems depend on which axioms and which are axiom-free.

3.6 Implementation Status

The following table distinguishes what is implemented, formally proven, axiomatized, and empirically tested:

Component	Implemented	Formally Proven	Axiomatized	Empirically Tested
Montgomery arithmetic	Yes	Yes (Lean 4)	—	Yes (104 vectors)
NTT butterfly (scalar)	Yes	Yes (Lean 4)	—	Yes
NTT stage decomposition	Yes	Yes (Lean 4)	—	Yes
AVX-512 NTT	Yes	—	3 axioms	Yes (differential)
Categorical backbone	Yes	Yes (Lean 4)	2 engineering axioms	—
FRI prover (oracle-layer)	Yes	—	—	Yes
FRI verifier (oracle-layer)	Yes	—	—	Yes
FRI bridge (delta_fri)	Scaffold	—	—	—
Sumcheck prover	Yes	—	—	Yes
Sumcheck verifier	No	—	—	—
DEEP-ALI prover	Yes	—	—	Yes
DEEP-ALI verifier	Partial	—	—	—
On-chain anchor (Sepolia)	Yes	—	—	Yes
On-chain FRI verifier	Scaffold	—	—	—
Proof serialization	Partial	—	—	—
Artifact-to-proof binding	No	—	—	—

3.7 Test Vectors

The 104 Rust test vectors cover: boundary cases (zero, one, p-1, p/2), random inputs (proptest with 10,000 random pairs), round-trip NTT property tests (forward then inverse equals identity), and differential testing (AVX-512 output vs. scalar output across all tested inputs).

The 61 verification points in the NTT benchmark correspond to 61 stage-by-stage comparison checks across log2(n) stages for multiple transform sizes (n = 2^5 through 2^10), verifying that the reference, scalar, and AVX-512 backends produce identical output at each stage, not only at the final output.

4. Security Analysis

4.1 ARCHER Methodology

The ARCHER (Adversarial Discovery) methodology is a structured approach to finding security issues at system boundaries. Rather than searching for specific attack patterns, ARCHER searches for cases where individually ordinary operations compose into unexpected behavior, where apparently irrelevant variables become relevant through interaction, and where assumptions about isolation, equivalence, reachability, harmlessness, or irrelevance fail.

The core loop is: Generate -> Perturb -> Execute -> Observe -> Compare -> Hypothesize -> Falsify -> Reproduce -> Report. The methodology explicitly requires falsification: every hypothesis must be tested against alternative explanations (ordinary behavior, measurement artifact, timing, state contamination, environmental variation, implementation defect, coincidence, alternative causal mechanism).

4.2 Findings

The ARCHER audit identified 36 findings across the protocol's repositories. 23 findings were fixed; 13 were documented as open engineering boundaries or design limitations. All findings were at system boundaries — configuration defaults, input validation, placeholder code, serialization, or interface contracts. No defects were identified in the tested portions of the cryptographic core (FRI commit/query/verify, Merkle commitments, Montgomery arithmetic) during the ARCHER audit. This result does not constitute a cryptographic security proof or independent cryptanalysis.

Phase 1: P0 Baseline (Python) — Findings 1-12

Finding 1: authorized_root defaults to "/". The artifact resolver's authorized root directory defaulted to "/", effectively disabling symlink protection. Any symlink in the filesystem could resolve to an unauthorized location. **Fix:** default to None and fail closed if not configured.

Finding 2: verify_predecessor accepts null predecessor on non-genesis receipts. The predecessor verification function accepted a null predecessor hash for receipts that were not genesis entries, allowing chain forks to be silently accepted. **Fix:** reject null predecessors for non-genesis receipts.

Finding 3: Timestamp hardcodes millisecond component. The timestamp generation function hardcoded ".000Z", losing millisecond precision. Two events within the same second would have identical timestamps, undermining the custody chain's ordering guarantees. **Fix:** use real millisecond precision.

Finding 4: FilesystemSourceHandler accepts any path. When authorized_root defaulted to "/", path validation always passed. **Fix:** same as Finding 1 — authorized_root must be explicitly set.

Finding 5: Symlink validation TOCTOU edge cases. While os.path.realpath() resolves symlinks, the check against authorized_root happens after resolution. Full TOCTOU protection requires OS-level file locking. **Documented** as design boundary.

Finding 6: ContentAddressedSourceHandler doesn't verify hash after download. Content was trusted without independent hash verification. **Fix:** added post-download hash verification.

Finding 7: Receipt chain extendable without validation. The extend method accepted receipts without validating chain continuity. **Fix:** added chain continuity check.

Finding 8: Artifact metadata spoofable via path. The artifact path was stored without integrity binding. **Fix:** added path binding to content hash.

Finding 9: No signatures on receipts. Receipts contain hashes but no signatures. This is by design for the P0 baseline (trust model is local). **Documented** as design boundary — signatures are added in tscp-anchor's on-chain anchoring.

Finding 10: Float log2 for power-of-two check. Used math.log2(n).is_integer() which has floating-point precision issues for large n. **Fix:** changed to (n & (n-1)) == 0 integer check.

Finding 11: MLE evaluation uses binary index lookup. The MLE evaluation function only worked at Boolean points, not arbitrary field element points. **Fix:** changed to proper evaluate_mle() function using Lagrange interpolation.

Finding 12: BatchMerkle uses random Poseidon2 constants. Using Perm::new_from_rng() produced different hash functions on each call, causing every Merkle opening to fail. **Fix:** changed to default_babybear_poseidon2_16() for deterministic, fixed constants.

Phase 2: tscp-anchor Core (Rust/Solidity) — Findings 13-25

Finding 13: DEEP-ALI uses additive shift instead of multiplicative. DEEP-ALI uses z + shift (additive) while the constraint layer uses multiplicative shift. This inconsistency means DEEP-ALI's shifted evaluations may not match the constraint layer's expectations. **Documented** — requires design-level decision.

Finding 14: delta_fri_bridge always returns Ok(true). The FRI bridge unconditionally returns success without checking any proof data. Any proof is "verified" regardless of correctness. **Documented** — the real FRI implementation exists in fri_query.rs and should be used instead.

Finding 15: delta_fri_bridge commit() discards evaluation data. The commit method takes evaluation data but doesn't store it or produce a real commitment. **Documented** as scaffold.

Finding 16: DEEP-ALI prove() doesn't produce quotient proof. The prove method computes the quotient polynomial but doesn't produce a FRI commitment for it. **Documented** — quotient should flow into the oracle-layer FRI prover.

Finding 17: DEEP-ALI verify() doesn't verify the quotient. The verify method checks batch consistency but doesn't verify the quotient polynomial commitment. **Documented** — requires connecting DEEP-ALI to the real FRI verifier.

Finding 18: Top-level OWSL bridge doesn't verify content hash. The top-level OWSL bridge trusted the daemon's checksum_valid boolean without independently verifying the content hash. **Fix:** added verify_content_hash() method.

Finding 19: verify_golden() hardcodes test claim. The verify_golden() method checks self.claim == 294373 (Fibonacci terminal). **Documented** as test-only — production verification should use cryptographic proof checking.

Finding 20: Sumcheck has no verifier. The sumcheck_round() function computes the prover's round polynomial, but no sumcheck_verify() function exists. **Documented** as incomplete — implementing the verifier is a grant-funded task.

Finding 21: dispatch_event uses counter%2 for transition kind. Uses counter parity to determine ClaimCreated vs ClaimVerified, not the event's content. **Documented** as placeholder.

Finding 22: evaluate_mle is O(n*2^n) — exponential. The MLE evaluation function iterates over all 2^n Boolean points. For n=20, each evaluation requires ~20 billion operations. **Documented** as performance limitation.

Finding 23: Additive vs multiplicative shift inconsistency. DEEP-ALI uses additive shift while constraints use multiplicative shift. **Documented** — requires design-level decision.

Finding 24: Transcript may use incompatible Poseidon2 constants. The Transcript struct uses Poseidon2Config::new() which may produce different constants than default_babybear_poseidon2_16(). **Documented** with warning.

Finding 25: Two parallel FRI implementations. The oracle-layer contains a real, working FRI and a scaffold that always returns success. **Documented** — the scaffold should delegate to the real implementation.

Phase 3: tscp-verifier + poly_ir — Findings 26-30

Finding 26: Emitter hardcodes fiat_shamir_rounds to 12. Sets fiat_shamir_rounds: 12 instead of using the actual round count. **Fix:** now passes the actual count from the ProveResult.

Finding 27: verifier_unchanged always true. The Verification struct sets verifier_unchanged: true unconditionally without comparing binary digests. **Documented** with TODO.

Finding 28: public_inputs_hash is placeholder. Computes sha256("public_inputs") — hashing the literal string, not actual public input data. **Documented** as placeholder.

Finding 29: SerializableFriProof omits Merkle openings. Proof serialization includes roots, final value, and query indices, but omits the actual Merkle opening proofs. The proof digest is computed over an incomplete representation. **Documented** with warning.

Finding 30: SerializableFriProof uses debug format for roots. Merkle roots are serialized using Rust's {:?} debug format, not canonical byte representation. The digest is non-reproducible across platforms or Rust versions. **Documented** with warning. Also added expression depth validation (max 100) to poly_ir.rs.

Phase 4: Remaining Crates — Findings 31-36

Finding 31: Foxtrot harness uses hardcoded trace commitment. The compute_trace_commitment method returns hardcoded bytes [height, width, 0xDE, 0xAD, 0xBE, 0xEF] — not a real Merkle commitment. **Documented** as placeholder.

Finding 32: Prover-server DEEP-ALI uses additive shift with unchecked cast. The evaluate_shifted method uses additive shift (z + shift) — same issue as Finding 23 — and casts shift from usize to u32 without overflow check. **Documented** with TODO.

Finding 33: Naive Lagrange interpolation is O(n^2). The interpolate_lagrange_naive function compounds the O(n*2^n) performance issue from Finding 22. **Documented** — production should

use FFT-based interpolation (O(n log n)).

Finding 34: poly_div doesn't check for zero denominator. Polynomial division didn't check if the denominator's leading coefficient was zero before inverting. **Fix:** added zero check.

Finding 35: SoundnessAccumulator not bound to Fiat-Shamir transcript. The soundness budget tracker is not cryptographically bound to the challenger. A malicious prover could reset the accumulator without affecting the challenge sequence. **Documented** with warning.

Finding 36: Incompatible Montgomery forms across NTT implementations. The zksha-rx implementation uses R=2^32 Montgomery form (u32, 16-lane SIMD) while tscp-pl-phase1 uses R=2^64 Montgomery form (u64, 8-lane SIMD). Data exchange requires conversion, which is sketched but not implemented. **Documented** as design inconsistency.

4.3 Pattern Analysis

Pattern	Findings
Configuration defaults	1, 4, 5
Input validation	2, 7, 8, 30, 34
Integrity/authentication	6, 9, 18, 27, 28, 29, 35
Interface contracts	10, 11
Documentation drift	12
Precision/representation	3, 10, 30, 36
Placeholder/scaffold	13, 14, 15, 21, 31
Protocol design	16, 17, 23, 32
Performance	22, 33
Duplicate code	25
Test-only in production path	19, 24
Hardcoded values	26, 28, 31

The findings cluster at system boundaries — interfaces between components, configuration defaults, and placeholder/scaffold code. The ARCHER audit examined the FRI, Merkle, and Montgomery arithmetic implementations and observed no fundamental cryptographic flaws in the tested portions. The oracle-layer FRI implementation (fri_query.rs) passed the ARCHER tests for Fiat-Shamir transcript reconstruction, Merkle verification, and fold consistency. This testing does not establish cryptographic soundness.

4.4 Remaining Boundaries

Five formal axioms (Section 3.5) remain open. These are explicitly documented engineering boundaries, not hidden assumptions. The protocol's security claim is conditional: *if* the axioms hold (backed by empirical evidence), *then* the formal guarantees follow (proven in Lean 4).

No external audit has been conducted. The ARCHER audit was performed by an AI agent (the author's assistant), which provides useful coverage but is not a substitute for independent human review. The protocol's own principle applies: a declared audit is an assertion, not a verification.

4.5 Binding Audit (P0-2)

A targeted binding audit examined the proof statement and public-input schema to determine whether the implementation implicitly establishes artifact-to-computation binding. The audit found that it does not.

What the proof system proves: The sumcheck protocol proves that the sum of the folded oracle over the Boolean hypercube equals the prover's claimed sum. The proof request takes two trace columns (col0, col1) and an alpha challenge as raw input. The proof envelope seals a claim value (currently the Fibonacci terminal: 294373) and the serialized proof payload.

What the on-chain anchor records: The TSCPAnchor contract records a batch hash and the committer's address. The batch hash is not cryptographically linked to the proof.

The gap: There is no step in the current implementation where the external artifact A is cryptographically bound to the computation output Output(C). The controlling conclusion is:

```
H(A) = h  does NOT imply  A = Output(C)
```

and we currently have no evidence establishing:

```
Verify(pi, C, h_A) => h_A = H(Output(C))
```

The claim-evidence register records:

C-005 (Bind): OPEN / UNVERIFIED

C-009 (A = Output(C)): UNVERIFIED

Transcript binding to public inputs and trace commitment is distinguished from artifact binding. On-chain artifact anchoring is explicitly distinguished from proof/output binding.

Implication: Computational binding is a protocol requirement not yet satisfied by the current implementation. The proof system proves properties about traces; it does not prove that an external artifact was produced by the claimed computation. Establishing this binding is the central implementation task. However, adding the artifact hash as a public input is necessary but not sufficient. The following seams must all be closed:

Binding seam	Required question
Canonicalization	Is the exact canonical byte representation inside the proved statement?
Serialization	Is serialization deterministic and unambiguous?
Hash domain	Is H domain-separated from unrelated commitments?
Public input	Is h_A actually consumed by the circuit constraints?
Computation identity	Does C identify the exact computation/version/configuration?

Binding seam	Required question
Input commitments	Are computation inputs cryptographically committed?
Execution commitment	Is the proved trace tied to those inputs and that computation?
Output semantics	Does the circuit's Output(C) correspond to the externally resolved artifact bytes?
Transcript	Are all binding values included before challenge derivation?

An accepting proof must establish the entire relation, not merely that h_A appears in the proof envelope. Making this gap explicit, rather than assuming it is implicitly satisfied, is a contribution of this paper.

5. Security Claims and Assumptions

5.1 Security Claims

Subject to the stated cryptographic, implementation, and machine-refinement assumptions, TSCP claims:

1. When all stages are implemented, a promoted artifact will have passed the specified custody predicates (Resolve, Observe, Bind, Independent Verify).
2. Artifact identity is established over canonical bytes.
3. When implemented, the binding proof will be independently checked against a reconstructed Fiat-Shamir transcript.
4. Failed custody predicates cannot result in promotion.
5. Certified bridges preserve the specified admissibility, preservation, and reflection properties.

5.2 Non-Claims

TSCP does not currently claim:

1. That the current implementation cryptographically binds an external artifact to its claimed computation — the proof system proves trace properties (sumcheck over the folded oracle), but does not yet establish Bound(A, C, W). This is the central open requirement (Section 4.5).
2. That arbitrary computation is proven correct — even with binding, the proof establishes a relation between an artifact and a claimed computation, subject to the proof system's assumptions.
3. That the current on-chain verifier provides production-grade FRI verification — it is a scaffold with placeholder functions.
4. That the AVX-512 implementation is fully mechanically verified — three machine refinement axioms remain open.
5. That the system has received an independent security audit — the ARCHER audit was performed by an AI agent, not an independent party.
6. That provenance metadata alone establishes semantic truth — the binding proof establishes cryptographic binding, not that the claimed computation is semantically meaningful.
7. That the sumcheck verifier is complete — it is not yet implemented.
8. That the proof serialization is canonical or reproducible — Merkle openings are omitted and debug-format encoding is used (Findings 29, 30).

6. Related Work

6.1 ZK Proof Systems

Plonky3 [Polygon, 2024] provides the cryptographic primitives (BabyBear field, Poseidon2, Merkle MMCS) used by TSCP. TSCP does not modify Plonky3's cryptographic core; it builds a protocol layer on top. RISC Zero [2022] and StarkWare [StarkWare, 2018] provide general-purpose zkVMs that can prove arbitrary computation. TSCP is complementary: it could sit on top of any proof system to add custody verification.

6.2 Software Supply Chain Security

SLSA [Google, 2021] and SBOM [NTIA, 2021] address software supply chain integrity by tracking composition and build provenance through authenticated attestations and trusted build infrastructure. in-toto [Torreira et al., 2019] defines a framework for cryptographically attesting to the sequence of steps in a software supply chain, binding artifact subjects to cryptographic digests and recording how artifacts were produced. These systems provide verifiable provenance through trusted builders, attestations, and policy enforcement.

TSCP investigates a different and complementary property: cryptographically verifying the relation between an artifact and a claimed computation without requiring the verifier to reproduce the computation or trust the build platform. SLSA and in-toto establish provenance through authenticated attestations and trusted infrastructure; TSCP seeks to establish a cryptographic binding between artifact and computation via ZK proofs. These approaches are complementary, not competing. A system could use SLSA/in-toto for supply chain attestation and TSCP for computational binding.

Important caveat: The cryptographic binding that distinguishes TSCP from these systems is currently an open requirement, not an established property (Section 4.5). The novelty of TSCP's contribution depends on establishing this binding.

6.3 Proof-Carrying Data and Recursive Composition

Proof-carrying data (PCD) [Chiesa and Tromer, 2010] and incremental verifiable computation (IVC) provide mechanisms for efficiently verifying distributed computations through recursive proof composition. Systems like HyperNova [Kothari and Setty, 2023] provide folding machinery for incremental computation, and PCD frameworks explicitly support recursive composition of proofs across computation steps.

TSCP's categorical backbone composition theorem is related but distinct. Whereas PCD/IVC frameworks focus on recursive composition of computational proofs, the reviewed prior art predominantly addresses translation or composition within formal proof ecosystems (e.g., OpenTheory, MMT, Dedukti) or within cryptographic proof systems; the review did not identify a system that combines formal proofs, runtime evidence, and empirical results as heterogeneous verification types under the same custody/promotion mechanism. The composition theorem proves that certified bridges preserve admissibility, preservation, and reflection — structural properties of verification systems, not computational claims about specific executions.

The novelty boundary is important: existing PCD/IVC systems already provide sophisticated composition of computational proofs. TSCP's contribution, if the binding is established, would be the custody-state composition — the explicit separation of identity, observation, binding, verification, and promotion into a state machine with fail-closed promotion semantics — rather than the composition mechanism itself.

6.4 AI Output Verification

AI watermarking [Kirchenbauer et al., 2023] and detection methods address identifying AI-generated content. They do not verify the production process. TSCP verifies provenance — the chain of computations that produced an artifact — not just content identity. However, this provenance verification is currently an architectural requirement, not an established implementation property.

6.5 Formal Verification of Cryptographic Protocols

Projects like fiat-crypto [Erbsen et al., 2019] and EverCrypt [Bhargavan et al., 2020] demonstrate that cryptographic implementations can be formally verified end-to-end. TSCP's Lean 4 verification follows a similar approach, with explicitly documented engineering boundaries where machine-level verification is not yet available. The mathematical layer (Montgomery arithmetic, NTT butterfly, stage decomposition) is fully proven; the machine refinement layer (AVX-512 intrinsics) has three open axioms backed by empirical testing.

7. Discussion

7.1 Custody Verification as a Primitive

The custody pipeline is a new primitive: a structured process for verifying artifact provenance using ZK proofs. It is not a proof system (it does not prove computations) and not a supply chain tool (it does not track composition). It is a protocol for verifying that an artifact's claimed history is real.

The five stages are designed to be independently useful: a system could implement only Resolve and Observe (audit logging), add Bind (proof generation), add Independent Verify (proof checking), and finally Promote (on-chain anchoring). Each stage adds a stronger guarantee.

7.2 Generalizability

TSCP is implemented on Plonky3, but the protocol is not tied to a specific proof system. The categorical backbone's Bridge abstraction allows different proof systems to be connected: a bridge from a Plonky3 universe to a RISC Zero universe would allow custody verification across proof systems, with the composition theorem guaranteeing that verification properties are preserved.

7.3 Limits of Self-Verification

The ARCHER audit was performed by an AI agent. This is useful — it found 36 real issues — but it is not independent. A compromised AI assistant could suppress findings. The protocol's own principle applies: a declared audit is an assertion, not a verification. An AI audit yielding a finding is not equivalent to an independent audit yielding a verified security property. That distinction should become a first-class part of the protocol: external review is necessary for the same reason that independent verification is necessary in the custody pipeline.

7.4 Epistemic Scope

The protocol's principle — a declared hash is an assertion, not a verification — generalizes beyond hashes to any asserted mapping. An acronym expansion, a provenance claim, a security audit — all are assertions until independently verified. The general rule: coherence is not evidence, a generated explanation is not recovered provenance, and internal consistency is not external authority. The custody pipeline enforces this for artifact provenance; the same epistemic boundary applies to the protocol's own claims, which is why the evidence-to-prose audit (Stage 2) and adversarial review

(Stage 3) were necessary.

8. Conclusion and Future Work

TSCP provides a structured protocol for cryptographic custody verification of artifacts produced by multi-agent computation pipelines. Its categorical formal backbone proves that verification properties compose correctly, its experimental ZK implementation provides concrete proof generation and partial verification, and its NTT optimization improves the execution speed of the optimized NTT kernel. The ARCHER audit demonstrates that the protocol's security can be tested systematically and that its boundaries can be hardened.

The central claim of TSCP's contribution is not "we built a better ZK system." It is: cryptographic artifact identity, computational correctness, and provenance are distinct properties, and a system should not promote an artifact from assertion to verified custody merely because its hash or provenance metadata was declared. TSCP makes those transitions explicit and independently checkable. The binding audit (Section 4.5) demonstrates this concretely: the implementation proves trace properties but does not yet bind the external artifact to the computation output. Making this gap visible — rather than assuming it is implicitly satisfied — is the protocol's first application of its own principle.

Future work:

1. **Establish artifact-to-proof binding** — commit the artifact hash as a public input in the proof statement, so that Verify(pi, C, h_A) => h_A = H(Output(C)). This is the central open requirement (Section 4.5).
2. **Discharge the formal axioms** using machine-level verification (Aeneas, Kraken) to complete the formal verification chain from hardware to mathematics.
3. **External audit** by an independent cryptographer.
4. **Complete the sumcheck verifier** and connect the DEEP-ALI quotient to the oracle-layer FRI prover.
5. **Production FRI verifier** — replace the on-chain scaffold with a real FRI verifier to complete the custody pipeline's promotion stage.
6. **Canonical proof serialization** — include Merkle openings and use canonical byte encoding for reproducible digests (Findings 29, 30).
7. **Multi-proof-system support** — implement bridges to additional proof systems (RISC Zero, StarkWare) to demonstrate generalizability.
8. **Define an interoperable custody receipt format and verification API** — specify artifact, canonical representation, artifact digest, computation identity, input commitments, execution commitment, binding proof, verifier result, custody predecessor, verification environment, and promotion record. This is a concrete milestone toward eventual standardization.

References

[1] Polygon. Plonky3: A toolkit for building ZK proof systems. 2024. [2] StarkWare. StarkWare and STARKs. 2018. [3] RISC Zero. RISC Zero zkVM. 2022. [4] Google. SLSA: Supply-chain Levels for Software Artifacts. 2021. [5] NTIA. Software Bill of Materials. 2021. [6] Kirchenbauer, J. et al. A Watermark for Large Language Models. 2023. [7] Erbsen, A. et al. Simple High-Level Code For Cryptographic Arithmetic. 2019. [8] Bhargavan, K. et al. EverCrypt: A Fast, Verified, Cross-Platform Cryptographic Provider. 2020. [9] Torreira, S. et al. in-toto: A framework to secure the integrity of software supply chains. 2019. [10] Chiesa, A. and Tromer, E. Proof-Carrying Data: Soundness and Concrete Efficiency. 2010. [11] Kothari, L. and Setty, S. HyperNova: Recursive arguments for

customizable constraint systems. 2023.

Appendices

A. Formal Backbone Structure (Lean 4)

The formal backbone consists of the following Lean 4 files:

- `TSCP_Formal_Backbone.lean` — Kernel, Universe, Bridge, BridgeCertificate, composition theorem
- `Core.lean` — BabyBear universe, NTT universe, bridges, engineering axioms
- `BridgePreservation.lean` — Bridge preservation properties

B. NTT Formal Verification (zksha-rx)

- `Butterfly.lean` — NTT butterfly operation
- `LanePacking.lean` — AVX-512 lane packing (zero axioms)
- `Machine.lean` — Machine refinement (3 open axioms)
- `MachineRefinements.lean` — Discharged refinements
- `Montgomery.lean` — Montgomery multiplication
- `NTT.lean` — NTT correctness
- `Stage.lean` — NTT stage decomposition

C. ARCHER Audit Results

Full audit results, including all 36 findings with observations, hypotheses, experiments, results, reproductions, falsification attempts, and claims, are available in the commit history of the `archer/fixes-p0-baseline` and `archer/fixes-tscp-anchor` branches on GitHub. The binding audit (Section 4.5) was conducted separately from the 36 ARCHER findings, as it examines the protocol's architectural completeness rather than implementation-level security.